1. Aim of the practical: Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views for each contest sorted by contest_id. Exclude the contest from the result.

2. Objective: retrieve contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views from your table, grouped by contest_id. Additionally, the query should exclude contests where all four of these sums are zero.

3. Commands/Steps:

Step-1: Created clg:
create database clg;

Step-2: Use this database:
Use clg;
Database changed

Setp-3: Created table contests:
ccreate table contests(contest_id int, hacker_id int, names varchar(20))

Steps-4: Created table challenges.
create table challenges(challenges_id int, college_id int)Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0

Steps-6: Created table view_stats
create table View_stats(challenges_id int, total_views int, total_unique_views int)

Step-: **Create table submission**
create table Submission(challeneges_id int, total_submissions int, total_accepted_submissions int)

5. Result:

Contests Table

Result Grid | Filter Rows: | Export:

contest_id	hacker_id	names
66406	17973	Varun
66558	79153	Vivaan
94828	80275	Anaya

College Table

college_id	contest_id
11219	66406
32473	66556
56685	94828
94828	94828

94828

Challenges Table

challenges_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table

challenges_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submissions Table

challeneges_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

Result:

```
SELECT c.contest_id, c.hacker_id, c.names,
COALESCE(SUM(s.total_submissions), 0)
AS total_submissions, COALESCE(SUM(s.total_accepted_submissions), 0)
AS total_accepted_submissions, COALESCE(SUM(v.total_views), 0)
AS total_views, COALESCE(SUM(v.total_unique_views), 0)
AS total_unique_views FROM contests c LEFT JOIN colleges co
ON c.contest_id = co.contest_id LEFT JOIN challenges ch
ON co.college_id = ch.college_id LEFT JOIN View_stats v
ON ch.challenges_id = v.challenges_id LEFT JOIN Submission s
ON ch.challenges_id = s.challeneges_id
GROUP BY c.contest_id, c.hacker_id, c.names
HAVING SUM(s.total_submissions) > 0 OR SUM(s.total_accepted_submissions) > 0
OR SUM(v.total_views) > 0 OR SUM(v.total_unique_views) > 0 ORDER BY c.contest_id;
```

Query 1 | SQL File 3*

Limit to 1000 rows

```sql
GROUP BY
c.contest_id, c.hacker_id, c.names
HAVING
SUM(s.total_submissions) > 0 OR
SUM(s.total_accepted_submissions) > 0 OR
SUM(v.total_views) > 0 OR
SUM(v.total_unique_views) > 0
ORDER BY
c.contest_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

contest_id	hacker_id	names	total_submissions	total_accepted_submissions	total_views	total_unique_views
66406	17973	Varun	222	78	238	122
94828	80275	Anaya	150	38	82	30

6. Learning outcomes (What I have learned):

a) Understanding SQL Joins different types of joins (e.g., LEFT JOIN) to combine data from multiple tables and relate them based on common keys.

b) Working with aggregate functions such as SUM() to calculate totals across multiple related records,

c) Understanding the importance of the HAVING clause to filter groups of data based on aggregate values